Build pipelines

Databricks pipeline engineer

A1 Code structure

A1-L1 Single-purpose functions and clear names

Each strong function passes the one-sentence test. The weak one needs "and" four times.

The five jobs a pipeline function does, one at a time	Read, transform, validate, write, log. An orchestrator calls them in order.
The one-sentence test	Describe the function without using "and". If you need "and", split it.
Function, class and constant naming	check_table_health, CheckResult, MAX_WEIGHT_CV.
What does a get_ or check_ prefix promise?	No side effects. The function reads and returns. It never writes or deletes.

A1-L2 Type hints and docstrings a reviewer can trust

The strong version tells mypy, the reviewer and the caller the same story. The weak version tells three different ones.

```python
def weighted_share(df: DataFrame, flag_col: str) -> float | None:
    """Weighted share of rows where flag_col is 1.

    Args:
        df: Respondents with final_weight.
        flag_col: 0/1 column to share.

    Returns:
        Share between 0 and 1, or None when the total weight is 0.

    Raises:
        ValueError: If flag_col is missing from df.
    """
```

Optional parameter that defaults to None	param: str | None = None. Never param: str = None.
A fixed set of string values	Literal["PASS", "WARN", "FAIL"] from typing.
Google docstring sections	One-line summary, then Args, Returns, Raises.
Why merge_keys: list[str] and not list?	mypy then rejects a string, which Python would split into characters.

A1-L3 Config over hardcoded values, and PEP 8 with ruff

The strong version runs unchanged in dev and prod. The contract file decides the table and the limits.

What belongs in configuration?	Anything that changes between environments, trackers or contract versions: names, maps, thresholds, rules.
When do you validate configuration?	Once, on load, before any data moves. Fail fast with the key's name.

Why not raw.get("catalog", "research_prod")?	A default hides a missing key and can point a run at production.
The two ruff commands for CI	ruff check . and ruff format --check .

A2 Medallion architecture

A2-L1 Bronze: keep every delivery exactly as received

The strong version keeps every byte the supplier sent, labels every row with its origin, and reads the file once.

The four rules of the bronze contract	Append only. As received. Audit columns. Grain: respondent per wave per delivery file.
The four bronze audit columns	ingestion_id, source_file, ingestion_timestamp, delivery_version.
Why no inferSchema in bronze?	It guesses types and can destroy values, such as the leading zeros in 00123.
Rows written, without a second read	DeltaTable.forName(spark, t).history(1) and operationMetrics["numOutputRows"].

A2-L2 Silver: canonical names, checked types, one row per respondent per wave

The strong version fails early and clearly on a changed delivery, and turns bad values into a number you can check instead of a crash or a silent NULL.

```
F.col("final_weight").try_cast("double")
```

What does withColumnRenamed do with a missing column?	Nothing, silently. Check the source columns yourself and raise one error listing every missing one.
cast on Spark 4.0 (Runtime 17.3 LTS) with a malformed value	Raises CAST_INVALID_INPUT, because ANSI mode is on by default.
Safe cast for silver	F.col(c).try_cast(t), then count the new NULLs and check them against a threshold.
Silver dedup key	The full grain: respondent_id and wave_id.

A2-L3 Gold: state the grain, group by all of it, guard every ratio

The strong version cannot merge two trackers, reports a true zero for zero reach, and never divides by nothing.

```
COMMENT ON TABLE research_gold.nat_metrics IS 'Grain: tracker x wave x age_group x gender x region';
ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('grain' = 'tracker x wave x age_group x gender x region');
```

Where do you record a gold table's grain?	In the function docstring and on the table: COMMENT ON TABLE or a custom property such as 'grain'.
sum over no matching rows	Returns NULL, not 0. Wrap it: F.coalesce(F.sum(...), F.lit(0)).
Safe ratio in PySpark	F.when(base > 0, part / base). NULL when the base is 0.

Why is SET TBLPROPERTIES ('owner' = ...) wrong?	owner is reserved. Spark 4.0 raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY. Use ALTER TABLE ... OWNER TO.

A3 Delta Lake

A3-L1 Idempotent MERGE, and why you deduplicate the source first

The strong version is idempotent for any source: repeated keys, a retry or a full redelivery. The weak version works only while the supplier never repeats a row.

When is a pipeline idempotent?	When running it twice with the same input leaves the table as one run would.
Two conditions for an idempotent Delta MERGE	The ON condition covers the full grain, and the source has at most one row per key.
What happens when two source rows match one target row?	The MERGE fails with DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE.
Why not dropDuplicates() before a MERGE?	It keeps an arbitrary row per key. Use row_number() over a window ordered by version, newest first.

A3-L2 Schema enforcement is on by default. Evolve on purpose.

Schema changes become decisions a person reviews, not accidents a delivery causes.

Delta append with an extra column, no options	Fails with a schema mismatch. Enforcement is on by default.
MERGE with updateAll and an extra source column, no evolution	Succeeds and silently ignores the extra column. Only your own column check catches it.
Evolve one write on purpose	Append: option mergeSchema. MERGE: .withSchemaEvolution(). Planned: ALTER TABLE ADD COLUMN.
Why not set autoMerge.enabled for the session?	It turns evolution on for every MERGE in the job, including ones you did not intend.

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM

History turns a data incident into a two-minute restore. Keep it long enough to be useful.

Read an old Delta version	VERSION AS OF n or TIMESTAMP AS OF ts in SQL; option versionAsOf in PySpark.
What does RESTORE do to history?	Adds a new version with the old content. Nothing is deleted.
Default retention windows	Deleted files 7 days (delta.deletedFileRetentionDuration). Log 30 days (delta.logRetentionDuration).
Why is VACUUM ... RETAIN 0 HOURS dangerous?	It deletes the files that time travel and RESTORE need, and it breaks readers still using them.

Databricks' recommendation for layout and maintenance on UC managed tables	Liquid clustering (CLUSTER BY or CLUSTER BY AUTO) with predictive optimization.

A4 Unity Catalog

A4-L1 The three-level namespace, one catalog per environment

The strong version reads exactly the table its config names, whatever the session's current catalog is.

```
research_prod.research_silver.nat_respondents
```

```
def table_name(catalog: str, layer: str, tracker: str) -> str:
    suffix = {"bronze": "raw", "silver": "respondents", "gold": "metrics"}[layer]
    return f"{catalog}.research_{layer}.{tracker}_{suffix}"
```

Unity Catalog name format	catalog.schema.table, for example research_prod.research_silver.nat_respondents.
What does each level hold in this project?	Catalog: environment. Schema: layer or governance. Table: tracker at a grain.
Why never write a two-part table name in pipeline code?	It resolves against the session's current catalog, which varies by cluster, notebook and job.
Where do table names come from?	One function built from config, such as table_name(catalog, layer, tracker).

A4-L2 Grant to groups, grant the least, grant at the right level

Three rules: groups not users, schemas not tables, SELECT for people and MODIFY for pipelines.

```
GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
  `research_analysts`;
SHOW GRANTS ON SCHEMA research_prod.research_gold;
```

Privileges needed to read a table	USE CATALOG on the catalog, USE SCHEMA on the schema, SELECT on the table or schema.
Why grant SELECT on the schema rather than the table?	Schema grants are inherited by every current and future table in it.
Who gets MODIFY on production silver and gold?	The pipeline service principal. People get SELECT.
Why never grant to a user account?	User grants outlive the reason for them. Group membership is managed in one place.

A4-L3 Row filters and column masks in Unity Catalog

The strong script runs on Databricks, names groups instead of people, and uses three-part names throughout.

```
CREATE FUNCTION research_prod.research_governance.respondent_filter(tracker_id
  STRING)
RETURN is_account_group_member('research_approved_analysts') OR tracker_id = 'TRK-
  PUBLIC';
```

```

ALTER TABLE research_prod.research_silver.nat_respondents
SET ROW FILTER research_prod.research_governance.respondent_filter ON (tracker_id);
```

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode SET MASK research_prod.research_governance.mask_postcode;
```

Row filter in Databricks	A SQL UDF returning BOOLEAN, bound with ALTER TABLE t SET ROW FILTER f ON (col).
Column mask in Databricks	A SQL UDF taking the column value, bound with ALTER TABLE t ALTER COLUMN c SET MASK f.
is_account_group_member or is_member?	is_account_group_member: Unity Catalog groups are account-level. is_member checks workspace groups.
What is CREATE ROW ACCESS POLICY?	Snowflake syntax. Databricks rejects it. Reference defect 4.